

10PEARLS SHINE INTERNSHIP PROGRAM

AEROSENSE PEARLS AQI PREDICTOR

Name: Sadia Noreen

Data Science Intern

Mentors: Hafsa Imtiaz , Umema Ashar

Date of Submission: 02-Sep-2026

Contents

0.1	Executive Summary	2
0.2	Problem Statement & Motivation	2
0.2.1	Core Objectives	2
0.3	End-to-End System Architecture	3
0.3.1	Architecture Stages	3
0.4	Data Ingestion & Feature Engineering	3
0.4.1	Data Sources & Parameters	3
0.4.2	Feature Families	4
0.4.3	Leakage Prevention	4
0.5	Exploratory Data Analysis (EDA) Insights	4
0.6	Machine Learning Models & Benchmarking	5
0.6.1	Model Spectrum	5
0.6.2	Production Selection Rule	5
0.7	Model Explainability (XAI with SHAP)	6
0.8	Early Warning Alert System	6
0.9	API & Dashboard Serving Layer	6
0.9.1	FastAPI REST Service	6
0.9.2	Streamlit Web Application	7
0.10	MLOps, CI/CD & Automation	7
0.11	Submission Details	8

0.1 Executive Summary

AeroSense is a fully automated, production-oriented machine learning system that generates continuous three-day AQI forecasts and proactive pollution alerts for Lahore, Karachi, and Islamabad. Unlike reactive platforms that only report historical data, AeroSense features a decoupled MLOps pipeline that ingests live data, computes engineered features, benchmarks various forecasting models, and applies SHAP explainability. The resulting forecasts are exposed through a FastAPI backend and an interactive Streamlit dashboard. Furthermore, the system runs completely unattended via scheduled GitHub Actions workflows that refresh the feature store hourly and retrain the model daily.

0.2 Problem Statement & Motivation

Urban centres in Pakistan most notably Lahore, Karachi, and Islamabad experience severe seasonal smog and persistent air pollution driven by a combination of vehicular emissions, industrial discharge, agricultural crop-residue burning, and meteorological temperature inversions that trap pollutants close to ground level. During peak winter months (typically October through February), AQI readings in Lahore frequently cross into the Hazardous category (AQI greater than 300), placing the entire population at acute health risk.

0.2.1 Core Objectives

- **Multi-Horizon Forecasting:** Predict AQI at +24h, +48h, and +72h horizons with strict time-series separation between training and evaluation data to guarantee zero data leakage.
- **Statistical-to-Deep-Learning Benchmarking:** Compare a naive persistence baseline against a regularised linear model (Ridge Regression), a tree-based ensemble (Random Forest), and a sequence-based deep learning model (LSTM).
- **Model Explainability (XAI):** Translate opaque model internals into transparent, point-by-point feature contributions using SHAP, enabling end-users to understand the rationale behind each forecast.
- **Early Warning System:** Automatically trigger categorised public-health alerts before dangerous AQI thresholds are crossed, providing lead time for precautions.
- **Decoupled, Serverless MLOps:** Maintain continuous, unattended operation through scheduled GitHub Actions pipelines integrated with the Hopsworks Feature Store and Model Registry.

0.3 End-to-End System Architecture

AeroSense follows a decoupled three-tier MLOps architecture consisting of a Feature Pipeline, a Training & Registration Pipeline, and an Inference & Serving Pipeline. Each tier operates independently and communicates exclusively through the Hopsworks Feature Store and Model Registry.

0.3.1 Architecture Stages

Stage	Component	Function
1. Data Ingestion	AQICN API, OpenWeather API, Open-Meteo API	Collects live pollutant concentrations, live weather observations, and historical climate data for each target city.
2. Feature Engineering	Raw Ingestion Aggregator, Feature Engineering Engine	Aggregates raw records, computes lag/rolling/cyclical features, and writes results to the Feature Store.
3. Model Training	Supervised Split, Model Benchmark, Model Registry	Performs chronological train/test split, benchmarks candidate models, and registers the winning artefact.
4. Explainability	SHAP Explainer Pipeline	Computes global and per-prediction SHAP attributions for the registered model and stores them.
5. Inference	Inference Engine, Hazardous Alert Classifier	Generates forecasts for every city and classifies them against the public-health alert ladder.
6. Serving	FastAPI REST API, Streamlit Dashboard	Exposes forecasts, alerts, and model metadata to end users via a REST API and interactive web dashboard.

0.4 Data Ingestion & Feature Engineering

0.4.1 Data Sources & Parameters

AeroSense integrates real-time and historical telemetry across three external APIs, monitoring both pollutant concentrations and meteorological indicators:

- **Pollutants:** PM2.5, PM10, Ozone (O_3), Nitrogen Dioxide (NO_2), Sulphur Dioxide (SO_2), Carbon Monoxide (CO), and US-EPA standardised AQI.
- **Meteorological Indicators:** Temperature ($^{\circ}C$), Relative Humidity (%), Wind Speed (m/s), Wind Direction ($^{\circ}$), Atmospheric Surface Pressure (hPa), Precipitation (mm), and Cloud Cover (%).

0.4.2 Feature Families

Feature engineering logic is implemented in `ml/features/engineering.py` and categorized into four core groups:

Feature Category	Formulation	Domain Intuition
Lagged Features	AQI(t-1), AQI(t-3), AQI(t-6), AQI(t-24); PM2.5 lag series	Captures atmospheric auto-correlation and short-term momentum.
Rolling Statistics	Mean, std dev, and max over 3h, 6h, and 24h windows	Quantifies localized volatility and background pollutant accumulation.
Cyclical Temporal	sin/cos transforms of local hour, month, and wind direction	Preserves circular periodicity (e.g., transition from Dec to Jan).
Domain-Derived	1h/24h AQI deltas; PM2.5/PM10 ratio; stagnant-air indicator	Distinguishes smog from dust and highlights stagnant conditions.

0.4.3 Leakage Prevention

All grouping and windowed transformations are computed strictly per city (`cityid`), and supervised prediction

0.5 Exploratory Data Analysis (EDA) Insights

Exploratory analysis performed via `ml/analysis/eda.py` uncovered four critical patterns that shaped the modeling strategy:

1. **Severe City Disparities:** Lahore exhibits the highest average AQI and greatest variance, spending over 70% of winter hours in unhealthy-to-hazardous bands.

2. Seasonal Dynamics: A distinct pollution peak occurs from October to February due to crop-residue burning and low boundary-layer inversions.
3. Diurnal Rhythms: AQI follows a bimodal daily pattern, peaking during morning (07:00-09:00) and evening (18:00-22:00) rush hours.
4. Meteorological Correlation: AQI correlates negatively with wind speed and precipitation (natural cleansers) and positively with stagnant, high-pressure conditions.

0.6 Machine Learning Models & Benchmarking

All models are evaluated independently at horizons of +24h, +48h, and +72h using a chronological test split of the final 20% of each city’s data.

0.6.1 Model Spectrum

Model	Description	Configuration
Persistence Baseline	Naive forecast assuming future AQI equals current AQI.	No trainable parameters.
Ridge Regression	L2-regularised linear model with median imputation and scaling.	scikit-learn Ridge with cross-validated alpha.
Random Forest	Non-linear tree ensemble optimized for fast serverless inference.	120 trees, max depth 20, min leaf size 20.
LSTM (Deep Learning)	Recurrent neural network for multi-step sequence forecasting.	Input shape: (batch, 24, n_features).

0.6.2 Production Selection Rule

To optimize memory and compute footprint, AeroSense applies an efficiency-first selection rule: if a lighter-weight model achieves an RMSE within 2% of the top-performing model, the more efficient model is selected for deployment.

0.7 Model Explainability (XAI with SHAP)

To ensure complete transparency, AeroSense implements SHAP in `ml/pipelines/explain.py` across two levels:

- **Global Explanations:** Evaluates a representative background sample to produce summary impact distributions showing key drivers across the dataset.
- **Local Real-Time Explanations:** Evaluates current-hour features for a target city to output exact additive contributions, where predicted AQI equals a base value plus feature contributions.
- **Plain-Language Translation:** Translates raw technical metrics into citizen-friendly insights (e.g., “*calm wind raises AQI by 14.2 points*”), driving the Streamlit visual explanations.

0.8 Early Warning Alert System

Implemented in `ml/pipelines/alerts.py`, the automated alert system classifies predicted AQI values against a four-tier public-health threshold ladder:

Tier	AQI Threshold	Guidance
EMERGENCY	AQI > 300	Avoid all outdoor activity; run HEPA air purifiers indoors.
SEVERE	AQI > 200	Avoid outdoor exertion; high-risk groups should remain indoors.
HIGH	AQI > 150	Limit prolonged outdoor activity; wear N95 masks outside.
MODERATE	AQI > 100	Sensitive groups should reduce prolonged outdoor exertion.

0.9 API & Dashboard Serving Layer

0.9.1 FastAPI REST Service

Implemented in `api/main.py`, the REST API exposes core endpoints for system integration:

Endpoint	Method	Description
/health	GET	Liveness check; reports feature store status and timestamp.
/cities	GET	Lists covered cities with current observed AQI and metrics.
/current/{city_id}	GET	Detailed real-time atmospheric measurements for a city.
/forecast/{city_id}	GET	3-day forecast breakdown with top SHAP drivers.
/alerts	GET	Active public-health warnings for vulnerable cities.
/models	GET	Model benchmark leaderboard and registry metadata.

0.9.2 Streamlit Web Application

Implemented in `streamlit_app/app.py`, the dashboard provides a modern user interface featuring colour-coded severity badges, live weather cards, interactive 3-day Altair forecast curves, SHAP driver indicators, and live warning banners.

0.10 MLOps, CI/CD & Automation

All operational pipelines run automatically through scheduled GitHub Actions workflows:

Workflow	Schedule	Function
feature-inference.yml	Hourly	Ingests live data, updates store, runs predictions, and writes alerts.
training.yml	Daily at 00:30 UTC	Retrains models, benchmarks performance, registers top models, and refreshes SHAP.
ci.yml / benchmark.yml	On Push / PR	Enforces code quality, runs unit tests, and verifies benchmark performance.

0.11 Submission Details

- GitHub Repository: <https://github.com/Sadia446/pearls-aqi-predictor>
- Live Application: <https://aerosense-pearls-aqi-predictor.streamlit.app/>
- Submission Date: 02 September 2026